Learning Objectives

Learners will be able to…

- **Define abstract data types (ADT)**
- **Compare abstract data types and data structures**
- **Implement List ADT using a vector**
- **Apply different operations to list ADT**

Abstract Data Types

Understanding ADTs

In Computer Science, an **abstract data type (ADT)** is a theoretical concept of how to represent data. Most importantly, ADTs focus on the behavior of the data type. They are not concerned with how the data type is implemented. This allows you to abstract away the implementation of the data type. Instead, you can talk about how to use a graph, tree, or list. It does not matter what programming language you are using or how you implement the ADT.

As its name implies, ADTs offer a level of abstraction when coding. They encapsulate the internal structure of the data, exposing only those operations that are necessary. Because of this, the coder can focus on the interface (what the ADT can do) as opposed to the implementation (how it is done).

Think back to when you first learned about strings. The reference material introduced them as a sequence of characters. You also learned about what you could and could not do with strings. Copy and paste the code below to the panel on the left.

```cpp
#include <iostream>
#include <string>
#include <algorithm>

int main() {

    std::string str = "Far out in the uncharted backwaters of
        the unfashionable end of the western spiral arm of the
        Galaxy lies a small, unregarded yellow sun.";
    std::cout << (str.find("ded y") != std::string::npos) <<
        std::endl;
    std::string modifiedStr = str;
    std::replace(modifiedStr.begin(), modifiedStr.end(), 'a',
        '@');
    std::cout << modifiedStr << std::endl;

    return 0;
}
```

However, you did not learn about how all of this was done. Do strings in C++ use an array, a vector, or a linked list to represent the sequence of characters? In the code example above, which algorithm did C++ use to find the substring or replace all instances of the `a` character? From the point of view of the user, it does not matter.

What matters is the interface – those actions you can perform with strings. If, in a future version of C++, the implementation of the string class changes, the interface will not. You can still instantiate string variables and call string functions just as you always have.

ADT Importance

ADTs are crucial for several reasons:

- **Modularity**: They allow for greater modularity, as you can change the implementation without affecting the rest of the program.
- **Maintainability**: Code is easier to understand and maintain when you separate the interface from the implementation.
- **Reusability**: Because ADTs are not tied to specific implementations, they can be reused in different contexts.

ADT vs Data Structure

The table below summarizes the contrast between ADTs and data structures based on features like focus, implementation, user perspective, and advantages.

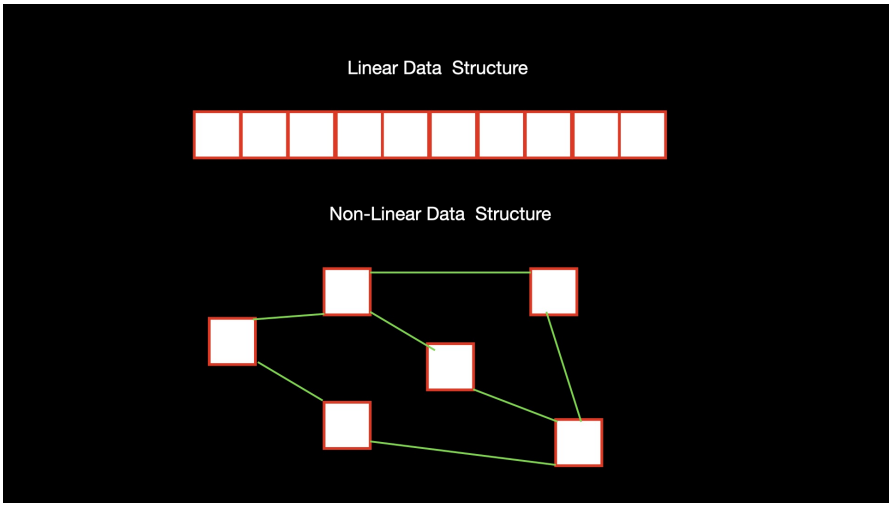
Feature	Abstract Data Type (ADT)	Data Structure
Focus	Behavior of the data	Concrete representation of the data
Implementation	Not specified	Specified
User perspective	What operations can be performed on the data	How the data is stored and organized
Advantage	More modular, easier to understand, more portable	Better performance

ADTs focus on the bigger picture (the what), while data structures focus on the details (the how). While ADTs offer several advantages over a concrete data structure, they often come at the cost of performance.

Lists

Linear Data Structures

Linear data structures are the most straightforward form of data structures, where elements are arranged sequentially. Each element has a unique predecessor and successor except for the first and last elements, respectively.



In C++, the most common examples of linear data structures are arrays and `std::vector`s. While there are distinct differences between these two structures, they share similarities at a fundamental level. Each structure represents a unified block of data, where the elements are ordered, allowing traversal of the structure and access to the data stored in each element.

Arrays and vectors are not the only data structures that exhibit these characteristics. Linked lists are another variation of a linear data structure. Like arrays and vectors, linked lists store data in an ordered manner, can be traversed, and allow access to their data. However, instead of being a unified block of data, linked lists are composed of separate nodes connected by pointers. There are even variations within the linked list family, such as singly linked lists, doubly linked lists, and circular linked lists. With so many different variations of linear data structures, how do you talk about these similar yet different structures?

List Abstract Data Type

Computer scientists use the term **list abstract data type (ADT)** to refer to this particular group of data structures. This abstraction allows us to focus on the common characteristics and operations without worrying about whether the structure is a vector, a linked list, or another type of list. The List ADT has a set of shared principles:

- **Order Preservation**: The list maintains the order of elements as they were inserted.
- **Sequential Access**: Elements in a list are accessed one after another, either from the beginning to the end or vice versa.
- **Variable Size**: Unlike arrays in C++, vectors and linked lists can dynamically change size, allowing for more flexible storage options.

The List ADT also has a shared set of basic operations:

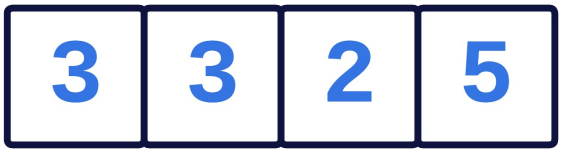
- **Insertion**: Adding a new element at a given position.
- **Deletion**: Removing an element from a specific position.
- **Traversal**: Moving through each element in the list.
- **Searching**: Finding the index or position of an element.

Lists offer a dynamic and flexible way to store data. They are excellent for scenarios where you need quick insertions and deletions, and where the size of the data set might change. Lists are a cornerstone in computer science and programming, serving as the foundational data structure for more complex types like stacks, queues, and even some graph representations.

Implementing the List ADT

Because the list ADT can be represented by several different concrete data structures, it is important to understand the consequences for selecting a particular implementation. Each data structure has its own advantages and disadvantages. The most common implementations are arrays and linked lists.

Array-Based Implementation



An array-based list uses an array to store its elements. The array may have a fixed size or can be dynamically resized.

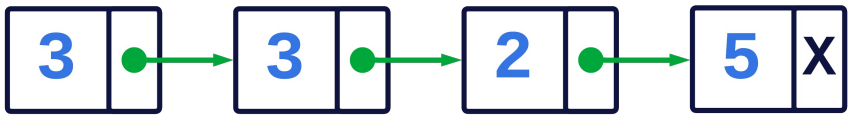
Strengths:

- Fast random access $O(1)$ to elements.
- Less memory overhead compared to linked lists.

Weaknesses:

- Insertions and deletions are generally slower $O(n)$ since elements may need to be shifted.
- Fixed-size arrays limit the list's size, while dynamic arrays incur occasional resizing costs.

Linked List-Based Implementation



A linked list-based list uses nodes, each containing a data element and a reference to the next node.

Strengths:

- Insertions and deletions are faster $O(1)$ if a reference to the insertion/deletion position is already available.

- No wasted space as the list grows or shrinks dynamically with the number of elements.

Weaknesses:

- Slower random access $O(n)$ to elements.
- Each node has additional memory overhead for storing the 'next' reference.

Choosing the Right Implementation

Using the information above, we can select the best implementation for some common scenarios:

- **Fixed vs. Dynamic**: If you know the maximum size of your list in advance, an array might be more efficient. For dynamic sizes, a linked list is generally better.
- **Access Patterns**: If your primary use case is random access, go for array-based lists. If you are frequently inserting and deleting elements, consider linked lists.
- **Memory Concerns**: If memory is a constraint, the less overhead of an array-based list may be beneficial.

Some advanced implementations combine features of both arrays and linked lists to offer a balanced set of advantages. For example, a dynamic array of linked lists or a linked list of sub-arrays.

Both array-based and linked list-based implementations have their pros and cons. The best choice depends on the specific requirements of your application. Understanding these trade-offs can help you make informed decisions and use the list ADT more effectively.

Arrays

Arrays in C++

Arrays are fundamental data structures in C++, offering fast access to elements through an index. They have been the dominant form of data structure that we have used so far. One of the defining characteristics of an array is its fixed size.

```
int integerArray[5];
std::string stringArray[] = {"John", "Jane", "Doe"};
```

This allows for predictable memory usage and fast speeds when accessing a stored value. However, arrays have some limitations. We are going to explore why we will not be using an array when implementing the List ADT.

Limitations

Arrays in C++ are homogeneous data structures, meaning they store elements of a single data type, whether it's primitive or user-defined types. Arrays have been the dominant form of data structures that we have seen up until this point.

Declaration and Initialization:

```
#include <iostream>
#include <string>

int main() {
    std::string pets[] = {"dog", "cat", "fish", "bird"};
    for (const auto &pet : pets) {
        std::cout << pet << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

However, bigger issues revolve around the fixed size of an array. Let's assume we want to add `"hamster"` to the array `pets`. There are no built-in methods to do this for us; everything must be done manually. Below is a method `add` that takes an array of strings and a string representing the new value to add to the array. The method returns a new array of strings with the added element.

```cpp
#include <iostream>
#include <string>

std::string* add(std::string arr[], int size, std::string val) {
    int newSize = size + 1;
    std::string* newArr = new std::string[newSize];

    for (int i = 0; i < size; i++) {
        newArr[i] = arr[i];
    }
    newArr[newSize - 1] = val;
    return newArr;
}

int main() {
    std::string pets[] = {"dog", "cat", "fish", "bird"};
    int size = sizeof(pets) / sizeof(pets[0]);
    std::string* newArray = add(pets, size, "hamster");

    for (int i = 0; i < size + 1; i++) {
        std::cout << newArray[i] << " ";
    }
    std::cout << std::endl;

    delete[] newArray;
    return 0;
}
```

Similarly, if we want to remove an element from the array, we need to write the logic ourselves. Below is a `remove` method that takes an array of strings and an integer representing the index of the element we want to remove.

```cpp
#include <iostream>
#include <string>

std::string* remove(std::string arr[], int size, int index) {
    int newSize = size - 1;
    std::string* newArr = new std::string[newSize];

    for (int i = 0, j = 0; i < size; i++) {
        if (i != index) {
            newArr[j] = arr[i];
            j++;
        }
    }

    return newArr;
}

int main() {
    std::string pets[] = {"dog", "cat", "fish", "bird"};
    int size = sizeof(pets) / sizeof(pets[0]);
    std::string* newArray = remove(pets, size, 2);

    for (int i = 0; i < size - 1; i++) {
        std::cout << newArray[i] << " ";
    }
    std::cout << std::endl;

    delete[] newArray;
    return 0;
}
```

Adding and removing elements from a list is a common task. With arrays in C++, these basic operations can feel a bit cumbersome. As such, we will not be using them when implementing the List ADT.

Vector

Vector in C++

The `std::vector` class is a part of the C++ Standard Library, and is present in the `vector` header file. It provides dynamic arrays and generally makes working with lists more enjoyable.

Let's recreate the operations from the previous page. Use the `push_back` function to add an element to the end of a vector. If we want to place an element at a specific location, we need to use the `insert` method. This method takes an iterator and the element to insert. If we want to add `rabbit` to the beginning of the vector of pets, use `pets.begin()` as the iterator for the first element of the vector.

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<std::string> pets = {"dog", "cat", "fish",
        "bird"};
    pets.push_back("hamster");
    pets.insert(pets.begin(), "rabbit");
    for (const auto& pet : pets) {
        std::cout << pet << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Removing elements is just as easy. The `pop_back` method removes the last element in the vector. The `erase` method removes a value at the given iterator. If we want to remove `3`, we can indicate this element with `numbers.begin() + 2`.

```
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5, 2, 6, 2, 7};
    numbers.pop_back();
    numbers.erase(numbers.begin() + 2);
    for (const auto& number : numbers) {
        std::cout << number << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

challenge

Try these variations:

- Remove all elements from a vector with the `clear` method.

```
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5, 2, 6, 2, 7};
    numbers.clear();
    for (const auto& number : numbers) {
        std::cout << number << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

- Remove all occurrences of `2` by combining `erase` and `remove`.

```
int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5, 2, 6, 2, 7};
    numbers.erase(std::remove(numbers.begin(),
        numbers.end(), 2), numbers.end());
    for (const auto& number : numbers) {
        std::cout << number << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

▼ What is happening?

> This is commonly called the “erase-remove idiom”. The `remove` method moves all of the elements with the value of `2` to the end of the vector. It then returns an iterator that points to the first element with the value of `2`. The `erase` method then deletes all elements from this iterator to the end of the vector.

We tend to think of a vector as a dynamic array, but that does not mean the syntax that works with arrays also works with vectors. Here are some common operations that are different with vectors.

The size of a vector is done with the `size` function, do not forget the parentheses.

```
int main() {
    std::vector<std::string> pets = {"dog", "cat", "fish",
        "bird"};
    std::cout << "The length of pets is: " << pets.size() <<
        std::endl;
}
```

You do not use square brackets when working with elements in a vector. To access a value, use the `at` method. To update the value of an element, you can directly assign it a new value

```
int main() {
    std::vector<std::string> pets = {"dog", "cat", "fish",
        "bird"};
    std::cout << "The first element is: " << pets.at(0) <<
        std::endl;
    pets.at(0) = "hamster";
    std::cout << "Vector after update: ";
    for (const auto& pet : pets) {
        std::cout << pet << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

challenge

Try this variation:

Use the `data` method to access the underlying array of the pets vector.

▼ Solution

Here is one possible solution.

```
int main() {
    std::vector<std::string> pets = {"dog", "cat", "fish",
        "bird"};
    std::string* petArray = pets.data();
    for(int i = 0; i < pets.size(); ++i) {
        std::cout << petArray[i] << " ";
    }
    std::cout << std::endl;
}
```

Implementing the List ADT with a vector offers many benefits like dynamic resizing. Most importantly, the built-in methods mean less functionality needs to be done manually. However, vectors are not without their own disadvantages. They are not synchronized, which means multiple threads can concurrently access the vector. This can lead to inconsistencies in the data. Vectors are generally slower than traditional arrays.

Vector Operations

Now that you're acquainted with arrays and vectors in C++ , let's delve deeper into some more operations you can perform with C++'s `std::vector` from the Standard Template Library (STL). We'll cover sorting, reversing, and sub-listing, along with their implications. We will tackle methods that define what the vector ADT can do, but not how it does it.

Sorting a Vector

Sorting is an essential operation in programming, and C++'s Standard Library makes it straightforward. Here's how to sort a vector of integers:

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // for sort

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    std::cout << "Before: ";
    for(int num : nums) std::cout << num << " ";
    std::cout << std::endl;

    std::sort(nums.begin(), nums.end());

    std::cout << "After: ";
    for(int num : nums) std::cout << num << " ";
    std::cout << std::endl;

    return 0;
}
```

Reversing a Vector

You can easily reverse a vector using the `reverse()` method from the algorithm library:

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // for reverse

int main() {
    std::vector<std::string> cities = {"Paris", "Tokyo",
        "Berlin"};
    std::cout << "Before: ";
    for(const auto& city : cities) std::cout << city << " ";
    std::cout << std::endl;

    std::reverse(cities.begin(), cities.end());

    std::cout << "After: ";
    for(const auto& city : cities) std::cout << city << " ";
    std::cout << std::endl;

    return 0;
}
```

Sublists

C++ doesn't provide a built-in method to create a subvector, but you can do so using iterators. Here's how:

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    std::cout << "Before: ";
    for(int num : nums) std::cout << num << " ";
    std::cout << std::endl;

    std::vector<int> numsSubList(nums.begin() + 1, nums.begin()
        + 4);

    std::cout << "After: ";
    for(int num : numsSubList) std::cout << num << " ";
    std::cout << std::endl;

    return 0;
}
```

Contains a Value

To check if a value exists in a vector, use the `std::find()` method, which returns an iterator to the element if found, or `end()` if not.

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // for find

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    int value = 5;
    auto it = std::find(nums.begin(), nums.end(), value);

    bool hasValue = (it != nums.end());
    std::cout << "Value " << value << " in list: " << (hasValue
        ? "Yes" : "No") << std::endl;

    return 0;
}
```

Find Index

Use `std::distance()` to find the index of a value in a vector. Note, this method will return the index for only the first occurrence of the value.

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // for find
#include <iterator>  // for distance

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    auto it = std::find(nums.begin(), nums.end(), 1);
    if (it != nums.end()) {
        std::cout << "Location of '1' is at index: " <<
        std::distance(nums.begin(), it) << std::endl;
    } else {
        std::cout << "Value not found." << std::endl;
    }
    return 0;
}
```

Vector Iterator

A vector iterator is an object from the `std` namespace. You can use it to iterate over a vector. The most common vector iterators are `begin()` (represents the first element) and `end()` (represents the last element).

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    for (auto it = nums.begin(); it != nums.end(); ++it) {
        std::cout << "Value: " << *it << std::endl;
    }
    return 0;
}
```

If you pass the iterator a value, it will create the iterator object starting at that index. So adding `2` to the `begin()` iterator will point to the third element in the vector.

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    for (auto it = nums.begin() + 2; it != nums.end(); ++it) {
        std::cout << "Value: " << *it << std::endl;
    }
    return 0;
}
```

You can even traverse over the vector in reverse order using the `rbegin()` and `rend()` iterators.

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    for (auto it = nums.rbegin(); it != nums.rend(); ++it) {
        std::cout << "Value: " << *it << std::endl;
    }
    return 0;
}
```

Remove on Condition

Use the “erase-remove” idiom to conditionally remove elements from a vector. Since we only want to remove an element if a certain condition is met, we are going to use the `std::remove_if` function. We pass `std::remove_if` iterators for the first and last elements in the vector. C++ will pass each element in the vector to the `isOdd` function, which returns `true` if the number in the vector is odd. Each time `isOdd` returns `true`, that element is moved to the back of the vector. The `erase` method then removes these elements from the vector.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

bool isOdd(int n) {
    return n % 2 != 0;
}

int main() {
    std::vector<int> nums = {3, 1, 4, 1, 5, 9};
    nums.erase(std::remove_if(nums.begin(), nums.end(), isOdd),
        nums.end());
    std::cout << "Even numbers: ";
    for(int num : nums) std::cout << num << " ";
    std::cout << std::endl;

    return 0;
}
```

challenge

Try this variation:

Assume the following vector:

```cpp
std::vector<std::string> names = {"Juan", "Douglas",
        "Juliette", "Clara"};
```

Implement a simple program that performs these operations:

- Sort a vector of names.
- Reverse the sorted vector.
- Create a subvector containing the first three names.
- Remove names if they start with the letter “J”.

▼ Solution

Here is one possible solution:

```cpp
#include <iostream>
#include <vector>
#include <algorithm> // for sort and reverse

int main() {
    std::vector<std::string> names = {"Juan", "Douglas",
        "Juliette", "Clara"};

    // Initial List
    std::cout << "Initial list: ";
    for(const auto& name : names) std::cout << name << " ";
    std::cout << std::endl;

    // Sort
    std::sort(names.begin(), names.end());
    std::cout << "After sorting: ";
    for(const auto& name : names) std::cout << name << " ";
    std::cout << std::endl;

    // Reverse
    std::reverse(names.begin(), names.end());
    std::cout << "After reversing: ";
    for(const auto& name : names) std::cout << name << " ";
    std::cout << std::endl;

    // Sublist
    std::vector<std::string> sublist(names.begin(),
        names.begin() + 3);
    std::cout << "Sublist (first 3 elements): ";
    for(const auto& name : sublist) std::cout << name << "
        ";
    std::cout << std::endl;

    // Remove names starting with 'J'
    names.erase(std::remove_if(names.begin(), names.end(),
        [](const std::string& name) {
        return name[0] == 'J';
    }), names.end());
    std::cout << "After removing names starting with 'J': ";
    for(const auto& name : names) std::cout << name << " ";
    std::cout << std::endl;

    return 0;
}
```

The vector in C++ offers a rich set of operations beyond the basic add, remove, and search. Understanding these operations and their implementations can greatly enhance your C++ programming skills, offering you a versatile toolset for data manipulation.

Formative Assessment 1

Formative Assessment 2